

Fast & Slow Pointer Pattern (Tortoise–Hare)

1. What is Fast & Slow Pointer?

Fast & Slow Pointer is a two-pointer technique where the slow pointer moves one step at a time and the fast pointer moves two steps at a time. It is mainly used to detect cycles and find middle elements in linked lists with $O(1)$ space.

2. When to Use This Pattern

- Detect cycle in a linked list
- Data structure is a **linked list**
- Find middle of a linked list
- Check palindrome linked list
- Happy Number and cyclic sequences
- Reordering/ rearranging linked list without extra space

3. Core Logic

Initialize both pointers at head. Move slow by one step and fast by two steps ie

```
slow = slow.next          fast = fast.next.next
```

If they ever meet, a cycle exists. If fast reaches null, no cycle exists.

4. Pseudocode Template

```
slow = head
fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
    if slow == fast:
        cycle found
```

5. Python Code Example

```
def hasCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

6. Diagram-Based flow

START



slow = head, fast = head



while fast != null and fast.next != null



slow = slow.next

fast = fast.next.next



if slow == fast → cycle found



END

7. Time & Space Complexity

Time Complexity: $O(n)$

Space Complexity: $O(1)$

8. Common Interview Problems

1. Linked List Cycle
2. Middle of the Linked List
3. Palindrome Linked List
4. Happy Number
5. Remove Nth Node from End

8. Interview Explanation Strategy

Explain brute force first, show extra space usage, then introduce fast & slow pointer as an optimized $O(1)$ space solution.

9. Common Interview Questions

Q1. Why does fast & slow pointer detect cycles?

→ Fast pointer laps slow pointer inside the cycle.

Q2. Why is space $O(1)$?

→ No extra data structure used.

Q3. Difference between Two Pointer & Fast-Slow Pointer?

→ Two Pointer = same speed

→ Fast-Slow = different speeds

Q4. Can it work without a cycle?

→ Yes (middle element problems)

Q5. Why not use HashSet for cycle detection?

→ HashSet uses extra space; fast-slow doesn't.